

Министерство образования и науки РФ
федеральное государственное автономное образовательное учреждение высшего образования
«Омский государственный технический университет»
Факультет Информационных технологий и компьютерных систем
Кафедра Информатики и вычислительной техники
Дисциплина Проектная деятельность

ЗАДАНИЕ
на выполнение курсового проекта
Студенту (ке) Шмидту Антону Владиславовичу Группа ИВТ-244
фамилия, имя, отчество полностью
Направление (специальность) 09.03.01 – Информатика и вычислительная техника
код, наименование
Тема проекта Основы применения шаблонов проектирования для улучшения структуры кода

Срок сдачи проекта (работы) на кафедру « 20 » 06 20 26 г.
Исходные данные к проекту (работе) _____

Содержание пояснительной записки (перечень подлежащих разработке вопросов)
Введение. 1. Теоретические основы шаблонов проектирования. 2. Практическое применение шаблонов для улучшения архитектуры кода. 3. Преимущества, риски и альтернативы. Заключение.

Перечень графического материала с указанием основных чертежей
и (или) иллюстративного материала нет

Методическая литература и иные информационные источники

1. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — СПб.: Питер, 2020. — 368 с.
2. Фримен Э., Робсон Э., Сьерра К., Бейтс Б. Паттерны проектирования (Head First Design Patterns). — СПб.: Питер, 2022. — 656 с.
3. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. — СПб.: Питер, 2021. — 352 с.
4. Фаулер М. Рефакторинг. Улучшение проекта существующего кода. — М.: Вильямс, 2019. — 448 с.

Дата выдачи задания « 2 » февраля 20 26 г.

Руководитель _____ ассистент Симоненко А.Е. _____
подпись ученая степень, звание, ФИО дата
Зав. кафедрой _____ К.т.н., доцент Грицай А.С. _____
подпись ученая степень, звание, ФИО дата

Задание принял к
исполнению студент (ка) _____ « 2 » 02 20 26 г.

подпись

Министерство образования и науки РФ
федеральное государственное автономное образовательное учреждение высшего образования
«Омский государственный технический университет»

Факультет (институт) Информационных технологий и компьютерных систем
Кафедра Информатики и вычислительной техники

КУРСОВОЙ ПРОЕКТ

по дисциплине Проектная деятельность
на тему Основы применения шаблонов проектирования для улучшения
структуры кода

Пояснительная записка

Шифр проекта 020-КП-09.03.01-129-ПЗ

Студента (ки) Шмидта Антона Владиславовича
фамилия, имя, отчество полностью

Курс 2 Группа ИВТ-244

Направление (специальность) 09.03.01 –
Информатика и вычислительная техника
код, наименование

Руководитель ассистент
ученая степень, звание
Симоненко А.Е.
фамилия, инициалы

Выполнил (а) _____
дата, подпись студента (ки)

К защите _____
дата, подпись руководителя

Выполнение и подготовка к защите КП (КР)	Защита КП (КР)	Итоговый рейтинг

Проект (работа) защищен (а) с оценкой _____

Омск 2026

Министерство образования и науки РФ

федеральное государственное автономное образовательное учреждение высшего образования
«Омский государственный технический университет»

ОТЗЫВ
на курсовой проект (работу)

Факультет информационных технологий и компьютерных систем

Кафедра Информатики и вычислительной техники

Дисциплина Проектная деятельность

Тема Основы применения шаблонов проектирования для улучшения структуры кода

Студент Шмидт Антон Владиславович
фамилия, имя, отчество полностью

Курс 2 Группа ИВТ-244

Руководитель ассистент Симоненко А.Е.
ученая степень, звание, ФИО

Содержание отзыва

Изучены архитектурные подходы к применению шаблонов проектирования в разработке: порождающие, структурные и поведенческие паттерны, а также их связь с принципами ООП (SOLID). Проведён анализ влияния различных шаблонов (на примерах Factory Method, Adapter, Strategy) и рассмотрены ключевые аспекты проектирования: снижение связности компонентов, масштабируемость архитектуры и риски избыточного усложнения кода (Overengineering).

Работа соответствует теме, содержит актуальные подходы и демонстрирует понимание материала.

Оформлена пояснительная записка.

Рейтинговые баллы за выполнение и подготовку к защите курсового проекта (работы)	
Заключение о допуске к защите	

Руководитель _____ Дата « » _____ 20 ____ г.
подпись

Оглавление

Введение	5
Глава 1. Теоретические основы шаблонов проектирования	7
1.1. История возникновения и определение паттернов	7
1.2. Связь шаблонов проектирования с базовыми принципами разработки	8
1.3. Классификация шаблонов проектирования	9
Глава 2. Практическое применение шаблонов для улучшения архитектуры кода	11
2.1. Порождающие шаблоны: изоляция процесса создания объектов (на примере «Фабричного метода»)	11
2.2. Структурные шаблоны: управление связями и интерфейсами (на примере «Адаптера» и «Фасада»)	12
2.3. Поведенческие шаблоны: оптимизация взаимодействия объектов (на примере «Стратегии» и «Наблюдателя»)	13
Глава 3. Преимущества, риски и альтернативы	15
3.1. Влияние паттернов на масштабируемость и командную разработку	15
3.2. Проблема избыточного проектирования (Overengineering) и антипаттерны...	16
Заключение	18
Список использованных источников	20

Введение

В современном мире разработка программного обеспечения характеризуется непрерывным ростом сложности систем и требований к их качеству. Сегодня от программиста требуется не просто написать код, который заставит приложение работать, но и обеспечить его долговечность. На первый план выходят такие характеристики, как читаемость, гибкость, легкость в поддержке и масштабируемость. Неграмотно спроектированная архитектура неизбежно приводит к накоплению «технического долга», усложняет командную работу и делает добавление новых функций ресурсозатратным или даже невозможным.

Одним из наиболее эффективных инструментов для решения этих проблем являются шаблоны (паттерны) проектирования — формализованные, многократно проверенные на практике способы решения часто возникающих задач при проектировании архитектуры программных систем. Шаблоны не являются готовым кодом; это концептуальные схемы, которые описывают суть проблемы и предлагают оптимальный путь ее решения. Их применение позволяет не изобретать «велосипед» при столкновении с типичной архитектурной задачей, а также формирует единый профессиональный словарь, который значительно упрощает коммуникацию внутри команды разработчиков.

Несмотря на широкую известность концепции, на практике программисты (особенно начинающие) часто сталкиваются с трудностями: от полного непонимания того, где уместен тот или иной паттерн, до слепого и избыточного их использования там, где достаточно простых решений (антипаттерн Overengineering). Именно это определяет высокую актуальность выбранной темы: понимание основ и границ применимости шаблонов проектирования является обязательным условием для создания качественного программного продукта.

Цель данного реферата — изучить теоретические и практические основы применения шаблонов проектирования, а также проанализировать их влияние на улучшение структуры и качества программного кода.

Для достижения поставленной цели были сформулированы следующие задачи:

- Рассмотреть историю возникновения шаблонов проектирования и их связь с фундаментальными принципами объектно-ориентированного программирования (в частности, SOLID);
- Изучить основную классификацию паттернов (порождающие, структурные, поведенческие);
- Проанализировать практическое применение наиболее востребованных шаблонов на конкретных примерах, демонстрирующих улучшение архитектуры;
- Выявить основные преимущества использования паттернов в командной разработке, а также риски, связанные с их избыточным применением.

Объектом исследования выступает процесс разработки и проектирования архитектуры программного обеспечения.

Предметом исследования являются шаблоны проектирования как инструмент оптимизации и улучшения структуры программного кода.

Глава 1. Теоретические основы шаблонов проектирования

1.1. История возникновения и определение паттернов

В инженерии программного обеспечения шаблон (паттерн) проектирования — это повторяемая архитектурная конструкция, представляющая собой решение типичной проблемы в рамках часто возникающего контекста проектирования. Важно понимать, что шаблон не является законченным фрагментом кода или библиотекой, которую можно просто скопировать и вставить в проект. Это, скорее, концептуальное описание, схема или чертеж, адаптируемые программистом под нужды конкретной системы.

Интересно, что концепция паттернов зародилась не в IT-индустрии, а в классической архитектуре и градостроительстве. В 1977 году архитектор Кристофер Александер опубликовал книгу «Язык шаблонов» (A Pattern Language), в которой описал типичные проблемы проектирования городов и зданий, предложив стандартизированные подходы к их решению — от расположения окон до высоты потолков. Александер заметил, что удачные архитектурные решения имеют общие черты, которые можно формализовать.

В конце 1980-х годов инженеры Уорд Каннингем и Кент Бек адаптировали идеи Александера для разработки программного обеспечения, применив их для проектирования пользовательских интерфейсов. Однако настоящий прорыв произошел в 1994 году с выходом фундаментальной книги «Паттерны объектно-ориентированного проектирования» (Design Patterns: Elements of Reusable Object-Oriented Software). Ее авторы — Эрих Гамма, Ричард Хелм, Ральф Джонсон и Джон Влиссидес — вошли в историю программирования как «Банда четырех» (Gang of Four, или сокращенно GoF). В этой книге авторы каталогизировали 23 классических шаблона, которые до сих пор остаются индустриальным стандартом.

С тех пор паттерны стали неотъемлемой частью профессионального языка разработчиков. Когда один программист говорит другому: «Здесь нужно использовать Фабрику», это экономит часы обсуждений, так как оба понимают, о какой именно архитектурной структуре идет речь, каковы ее плюсы, минусы и последствия применения.

1.2. Связь шаблонов проектирования с базовыми принципами разработки

Шаблоны проектирования не существуют в вакууме. Их эффективность обусловлена тем, что они являются практической реализацией фундаментальных принципов объектно-ориентированного проектирования (ООП). Чтобы структура кода была по-настоящему качественной, шаблоны необходимо рассматривать через призму принципов SOLID (акроним, введенный Робертом Мартином, объединяющий пять базовых правил).

Рассмотрим, как паттерны помогают соблюдать эти принципы:

- **Принцип единственной ответственности (Single Responsibility Principle):** Каждый класс должен иметь лишь одну причину для изменения. Шаблоны, такие как Фасад (Facade) или Заместитель (Proxy), помогают вынести сложную или побочную логику в отдельные классы, разгружая основные компоненты системы.
- **Принцип открытости/закрытости (Open/Closed Principle):** Программные сущности должны быть открыты для расширения, но закрыты для модификации. Это один из главных принципов, который реализуется с помощью паттернов. Например, шаблон Стратегия (Strategy) позволяет добавлять новые алгоритмы поведения в систему, не изменяя существующий код классов.
- **Принцип подстановки Барбары Лисков (Liskov Substitution Principle):** Объекты в программе должны быть заменяемыми на экземпляры их подтипов без изменения правильности выполнения программы. Большинство паттернов GoF строится на грамотном использовании полиморфизма и интерфейсов, что напрямую поддерживает этот принцип.
- **Принцип разделения интерфейса (Interface Segregation Principle):** Клиенты не должны зависеть от методов, которые они не используют. Паттерн Адаптер (Adapter) часто применяется для того, чтобы привести «толстый» и неудобный интерфейс к узкому и специфичному виду, требуемому конкретным клиентом.

- Принцип инверсии зависимостей (Dependency Inversion Principle): Модули верхних уровней не должны зависеть от модулей нижних уровней; оба должны зависеть от абстракций. Шаблон Абстрактная фабрика (Abstract Factory) является классическим примером реализации этого правила: система работает с интерфейсами создаваемых объектов, не привязываясь к их конкретным реализациям.

Таким образом, шаблоны проектирования — это инструментарий, позволяющий разработчику писать код, который естественно и органично соответствует лучшим практикам инженерии.

1.3. Классификация шаблонов проектирования

В зависимости от назначения и характера решаемых задач, 23 классических паттерна GoF принято делить на три большие группы. Такая классификация помогает разработчикам быстрее находить нужное решение, сужая область поиска.

1. Порождающие шаблоны (Creational Patterns) Эта группа паттернов отвечает за удобное и безопасное создание новых объектов. В базовом ООП объекты создаются напрямую вызовом конструктора (через оператор new). Это создает жесткую связь между кодом, который использует объект, и классом самого объекта. Порождающие шаблоны скрывают логику инстанцирования (создания) и делают систему независимой от того, как именно создаются, компонуются и представляются ее элементы. К ним относятся: Одиночка (Singleton), Фабричный метод (Factory Method), Абстрактная фабрика (Abstract Factory), Строитель (Builder), Прототип (Prototype).

2. Структурные шаблоны (Structural Patterns) Структурные паттерны решают вопросы о том, как из классов и объектов образовать более крупные и сложные структуры, сохраняя при этом их гибкость и эффективность. Они особенно полезны, когда две разные системы с несовместимыми интерфейсами должны работать вместе, или когда необходимо динамически добавлять функциональность объектам. К ним относятся: Адаптер (Adapter), Мост (Bridge), Компоновщик (Composite), Декоратор (Decorator), Фасад (Facade), Легковес (Flyweight), Заместитель (Proxy).

3. Поведенческие шаблоны (Behavioral Patterns) Эта категория шаблонов сфокусирована на алгоритмах и распределении обязанностей между объектами. Поведенческие паттерны не просто описывают структуры объектов или классов, они определяют паттерны коммуникации между ними. Их главная задача — минимизировать связанность компонентов при их взаимодействии и избавить код от сложных, запутанных условных конструкций. К ним относятся: Цепочка обязанностей (Chain of Responsibility), Команда (Command), Итератор (Iterator), Посредник (Mediator), Снимок (Memento), Наблюдатель (Observer), Состояние (State), Стратегия (Strategy), Шаблонный метод (Template Method), Посетитель (Visitor).

Глава 2. Практическое применение шаблонов для улучшения архитектуры кода

В то время как теоретические концепции предоставляют общую систему координат, истинная ценность шаблонов проектирования раскрывается при их непосредственном внедрении в практику разработки. Основная цель применения паттернов — борьба с жесткой зависимостью компонентов (tight coupling) и обеспечение гибкости системы при изменении требований.

В данной главе на конкретных примерах рассматривается, как представители трех основных групп шаблонов (порождающие, структурные и поведенческие) трансформируют проблемную, трудноподдерживаемую структуру кода в гибкую и масштабируемую архитектуру.

2.1. Порождающие шаблоны: изоляция процесса создания объектов (на примере «Фабричного метода»)

В традиционном объектно-ориентированном программировании создание экземпляра класса осуществляется с помощью оператора new (или его аналога в конкретном языке). Однако прямое конструирование объектов внутри бизнес-логики программы создает жесткую привязку к конкретным реализациям. Если в будущем потребуется заменить один класс другим или добавить поддержку новых типов объектов, разработчику придется изменять код во всех местах, где происходило инстанцирование, что прямо нарушает принцип открытости/закрытости (SOLID).

Для решения этой проблемы применяется шаблон Фабричный метод (Factory Method). Его суть заключается в том, что базовый класс делегирует создание объектов своим подклассам, определяя для этого специальный интерфейс.

Архитектурная проблема (до применения шаблона): Представим логистическую систему, которая изначально разрабатывалась только для автомобильных перевозок. В коде бизнес-логики (например, в классе ТранспортныйМенеджер) повсеместно создаются объекты класса Грузовик. При расширении бизнеса и необходимости добавить морские перевозки (класс Судно) выясняется, что вся система жестко завязана на специфику наземного транспорта.

Переписывание кода требует масштабного вмешательства и сопряжено с высоким риском внесения ошибок.

Решение и улучшение структуры (после применения шаблона): При внедрении «Фабричного метода» создается общий интерфейс Транспорт с методом доставить(). Классы Грузовик и Судно реализуют этот интерфейс. Базовый класс логистики объявляется как абстрактный (или создается интерфейс Логистика) и содержит абстрактный фабричный метод создатьТранспорт().

Теперь подклассы НаземнаяЛогистика и МорскаяЛогистика сами решают, какой именно объект конструировать. Основная бизнес-логика программы работает исключительно с абстракцией Транспорт.

Применение данного шаблона дает следующие преимущества для структуры кода:

1. Устранение жестких связей: вызывающий код полностью изолирован от конкретных классов создаваемых продуктов.
2. Упрощение расширения: для добавления нового вида транспорта (например, авиации) достаточно создать класс Самолет и подкласс АвиаЛогистика, не меняя ни одной строчки в уже существующем коде управления перевозками.

2.2. Структурные шаблоны: управление связями и интерфейсами (на примере «Адаптера» и «Фасада»)

Структурные шаблоны отвечают за то, как объекты соединяются друг с другом, минимизируя взаимное влияние при изменении отдельных частей.

Шаблон «Адаптер» (Adapter)

Часто в процессе разработки возникает необходимость интегрировать в систему стороннюю библиотеку, сервис или старый (легаси) код, интерфейс которого не совпадает с тем, что ожидает существующая архитектура.

Архитектурное решение: Шаблон «Адаптер» выступает в роли переводчика. Он оборачивает объект с несовместимым интерфейсом в специальный класс, который предоставляет клиенту стандартный и ожидаемый интерфейс. Таким образом, существующий код системы остается нетронутым, а вся логика трансформации данных или вызовов локализуется внутри одного класса-адаптера.

Это позволяет структурировать интеграцию внешних зависимостей и защитить ядро системы от изменений во внешних API.

Шаблон «Фасад» (Facade)

По мере роста программного комплекса количество внутренних классов, связей и подсистем неизбежно увеличивается. Клиентскому коду (например, пользовательскому интерфейсу или внешнему сервису) становится все сложнее взаимодействовать с такой системой, поскольку требуется знать внутреннее устройство десятков компонентов.

Архитектурное решение: Шаблон «Фасад» предоставляет единый, простой и высокоуровневый интерфейс к сложной подсистеме. Вместо того чтобы заставлять клиентский код вызывать методы классов ПроверкаЗапасов, ПроведениеОплаты, СлужбаДоставки и Логирование, создается один класс ОформлениеЗаказаФасад с единственным методом разместитьЗаказ().

Улучшение структуры проявляется в следующем:

- Снижается когнитивная нагрузка на разработчиков, использующих подсистему;
- Минимизируются зависимости между подсистемами, что позволяет изменять, оптимизировать или полностью переписывать внутренние компоненты фасада без вреда для остальной программы.

2.3. Поведенческие шаблоны: оптимизация взаимодействия объектов (на примере «Стратегии» и «Наблюдателя»)

Поведенческие шаблоны контролируют не просто структуру, а динамику выполнения программы — то, как объекты общаются и распределяют обязанности в процессе работы.

Шаблон «Стратегия» (Strategy)

Одной из самых распространенных проблем «плохого» кода является избыточное использование разветвленных условных операторов (if-else или switch-case) для выбора алгоритма поведения. Такой код быстро разрастается, становится монолитным и сложным для тестирования.

Архитектурное решение: Шаблон «Стратегия» определяет семейство схожих

алгоритмов, инкапсулирует каждый из них в отдельный класс и делает их взаимозаменяемыми прямо во время выполнения программы (*runtime*).

Например, в модуле расчета стоимости доставки вместо одного огромного метода со множеством условий создается интерфейс `СтратегияРасчета` и его конкретные реализации: `ОбычнаяДоставка`, `ЭкспрессДоставка`, `МеждународнаяДоставка`. Основной класс контекста лишь хранит ссылку на текущую стратегию и делегирует ей выполнение работы.

Структура кода значительно улучшается: исчезают громоздкие условные конструкции, а каждый алгоритм изолируется в собственном пространстве, что упрощает его изолированное тестирование (Unit-тестирование).

Шаблон «Наблюдатель» (Observer)

В событийно-ориентированных системах часто возникает ситуация, когда изменение состояния одного объекта требует оповещения множества других, связанных с ним объектов (например, изменение цены акции должно обновить графики, отправить уведомления пользователям и запустить торговых роботов). Привязывать все эти сервисы напрямую к объекту акции — значит создать архитектурный тупик.

Архитектурное решение: Шаблон «Наблюдатель» реализует механизм подписки. Объект, обладающий важным состоянием (`Издатель`), хранит список подписчиков (`Наблюдателей`) и предоставляет методы для их динамического добавления и удаления. При изменении состояния `Издатель` просто проходит по списку и вызывает у каждого `Наблюдателя` стандартизированный метод оповещения.

В результате структура кода приобретает слабую связанность (*loose coupling*): `Издатель` абсолютно ничего не знает о конкретных классах своих `Наблюдателей`, их внутреннем устройстве или количестве. Они связаны лишь общим интерфейсом, что позволяет динамически менять конфигурацию системы прямо в процессе ее работы.

Глава 3. Преимущества, риски и альтернативы

Рассмотренные в предыдущих главах примеры наглядно демонстрируют мощь шаблонов проектирования как инструмента для построения гибкой архитектуры. Однако, как и любая инженерная методология, применение паттернов не является универсальной панацеей (так называемой «серебряной пулей»). Их внедрение несет в себе не только неоспоримые преимущества, но и определенные риски, которые необходимо учитывать при проектировании программных систем.

3.1. Влияние паттернов на масштабируемость и командную разработку

Помимо чисто технических метрик (таких как снижение связности компонентов или инкапсуляция логики), шаблоны проектирования оказывают колоссальное влияние на процесс разработки как на социальное и организационное явление.

1. Формирование единого профессионального словаря. Это одно из самых недооцененных, но критически важных преимуществ. В промышленной разработке программы редко пишутся одним человеком. Когда в команде возникает архитектурная проблема, разработчикам не нужно тратить часы на рисование сложных UML-схем и объяснение принципов взаимодействия классов. Фраза «Давайте обернем этот класс в Декоратор» мгновенно передает всем участникам дискуссии исчерпывающую информацию о структуре будущего кода, его интерфейсах и способах интеграции.
2. Повышение читаемости и предсказуемости кода. Паттерны стандартизируют подходы к решению задач. Разработчик, открывающий чужой код и видящий в названии класса суффикс `...Factory`, `...Strategy` или `...Observer`, заранее понимает роль этого класса в системе и принципы его взаимодействия с другими компонентами. Это кардинально снижает порог вхождения в проект для новых сотрудников и ускоряет процесс проверки кода (Code Review).
3. Безопасная масштабируемость. Благодаря тому, что шаблоны (особенно структурные и поведенческие) строятся на принципах абстракции и полиморфизма, добавление нового функционала зачастую сводится к написанию новых классов, реализующих уже существующие интерфейсы, а

не к переписыванию старого, работающего ядра системы. Это минимизирует риск появления регрессионных багов (ситуаций, когда новое изменение ломает старый функционал).

3.2. Проблема избыточного проектирования (Overengineering) и антипаттерны

Несмотря на очевидную пользу, неграмотное использование шаблонов способно нанести архитектуре больше вреда, чем их полное отсутствие. Главной ловушкой, в которую часто попадают неопытные программисты, изучившие книгу «Банды четырех», является стремление применить паттерны везде, где это технически возможно.

Это явление получило название Overengineering (избыточное проектирование) — создание неоправданно сложного решения для простой задачи.

Внедрение любого шаблона неизбежно влечет за собой усложнение кодовой базы: появляются новые абстрактные классы, интерфейсы, слои делегирования. Если задача заключается в написании небольшого скрипта для одноразовой обработки файла, использование многоуровневой «Абстрактной фабрики» сделает код громоздким, трудночитаемым и сложным для отладки.

Проблема избыточного проектирования тесно связана с нарушением двух важных принципов разработки:

- KISS (Keep It Simple, Stupid): Система работает лучше всего, если она остается простой. Усложнять структуру следует только тогда, когда простые решения перестают справляться с требованиями.
- YAGNI (You Aren't Gonna Need It): Отказ от реализации функционала или архитектурных заделов «на будущее». Часто разработчики внедряют сложные шаблоны, предполагая, что система будет масштабироваться в определенном направлении, однако на практике бизнес-требования меняются, и созданная абстракция остается мертвым грузом, который лишь усложняет поддержку.

Альтернативный подход (Рефакторинг к паттернам) Современная парадигма инженерии программного обеспечения, популяризированная Мартином Фаулером,

гласит: разработку следует начинать с самого простого, процедурного или базового объектно-ориентированного кода. И только тогда, когда код начинает проявлять признаки «архитектурного запаха» (Code Smells) — например, дублирование логики, неконтролируемое разрастание условных операторов или слишком жесткая связность, препятствующая добавлению новой функции, — программист должен провести рефакторинг (перестройку) проблемного участка, внедрив подходящий шаблон проектирования.

Таким образом, шаблоны — это не строительные блоки, из которых нужно складывать приложение с самого начала, а скорее лекарство, которое следует применять строго по показаниям при выявлении архитектурных проблем.

Заключение

Подводя итог проведенному исследованию, можно с уверенностью утверждать, что шаблоны проектирования являются одним из важнейших фундаментов современной инженерии программного обеспечения. Эволюция подходов к разработке доказала, что написание кода, который просто выполняется машиной, больше не является самоцелью. В условиях постоянно меняющихся бизнес-требований на первый план выходит создание архитектуры, способной безболезненно адаптироваться к этим изменениям.

В ходе работы были решены все поставленные задачи. Во-первых, изучена теоретическая база паттернов. Показано, что они не являются искусственно придуманными правилами, а представляют собой кристаллизацию многолетнего опыта программистов и естественное продолжение фундаментальных принципов объектно-ориентированного дизайна (в частности, принципов SOLID).

Во-вторых, на примерах порождающих, структурных и поведенческих паттернов (таких как «Фабричный метод», «Адаптер», «Фасад», «Стратегия» и «Наблюдатель») было наглядно продемонстрировано, как именно улучшается структура кода. Применение этих подходов позволяет устранить жесткую связанность компонентов, скрыть сложность реализации за понятными интерфейсами и избавиться от громоздкой условной логики. Код становится модульным, тестируемым и легко расширяемым.

В-третьих, проанализировано влияние шаблонов на процесс разработки. Выявлено, что помимо технических преимуществ, паттерны формируют единый коммуникативный стандарт в команде, значительно ускоряя процессы обсуждения архитектуры и ревью кода. В то же время была обозначена критически важная проблема избыточного проектирования (Overengineering). Использование сложных абстракций ради самих абстракций, вразрез с принципами простоты (KISS) и целесообразности (YAGNI), приводит к деградации структуры проекта.

Главный вывод, который следует из данной работы: шаблоны проектирования — это мощный инструмент, а не самоцель. Качественная архитектура кода рождается не там, где применено максимальное количество паттернов из каталога

«Банды четырех», а там, где разработчик понимает суть проблемы и применяет паттерн точно, осознанно и только тогда, когда простые процедурные или базовые объектно-ориентированные решения перестают справляться с растущей сложностью системы.

Список использованных источников

1. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — СПб.: Питер, 2020. — 368 с.
2. Фримен Э., Робсон Э., Сьерра К., Бейтс Б. Паттерны проектирования (Head First Design Patterns). — СПб.: Питер, 2022. — 656 с.
3. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. — СПб.: Питер, 2021. — 352 с.
4. Фаулер М. Рефакторинг. Улучшение проекта существующего кода. — М.: Вильямс, 2019. — 448 с.